

reloj.js (StopwatchScreen)

Pantalla simple del lado del alumno: un cronometro (stopwatch, no un reloj de pared) para medir tiempo transcurrido, tipicamente usado para controlar los descansos entre series mientras se entrena. Es accesible desde la barra de navegacion inferior (activeScreen='clock').

1. Estado

```
const [elapsedTime, setElapsedTime] = useState(0); // milisegundos transcurridos
const [isRunning, setIsRunning] = useState(false); // si el cronometro esta corriendo
```

2. El corazon del componente: useEffect + setInterval (closures)

```
useEffect(() => {
  if (!isRunning) {
    return undefined;
  }

  const interval = setInterval(() => {
    setElapsedTime((currentTime) => currentTime + 1000);
  }, 1000);

  return () => clearInterval(interval);
}, [isRunning]);
```

Este bloque es una aplicacion directa del concepto de closures visto en clase:

- **Dependencia [isRunning]:** el efecto se vuelve a ejecutar cada vez que isRunning cambia (de false a true al tocar 'Iniciar', o de true a false al pausar).
- **Guard temprano:** si isRunning es false, el efecto no hace nada y retorna undefined (sin funcion de limpieza) — no queda ningun timer corriendo.
- **setInterval:** cuando isRunning pasa a true, se crea un intervalo que ejecuta cada 1000ms (1 segundo) una funcion que suma 1000ms a elapsedTime.
- **Funcion de actualizacion con closure:** se usa setElapsedTime((currentTime) => currentTime + 1000) en vez de setElapsedTime(elapsedTime + 1000). Esto es clave: el 'interval' es una closure que 'recuerda' el valor de elapsedTime que existia en el momento en que se creo el efecto. Como el intervalo no se vuelve a crear cada segundo, si se usara directamente 'elapsedTime + 1000' siempre sumaria sobre el mismo valor viejo capturado (el contador quedaria 'pegado' en 1 segundo repetido). Al pasarle una funcion al setter, React garantiza que el argumento (currentTime) sea siempre el estado mas reciente, sin importar que haya capturado la closure del intervalo en el momento de su creacion.
- **Cleanup (return () => clearInterval(interval)):** cada vez que el efecto se re-ejecuta (o el componente se desmonta), React ejecuta primero esta funcion de limpieza para el intervalo ANTERIOR. Asi, cuando el usuario pausa (isRunning pasa a false), el intervalo viejo se cancela y el nuevo useEffect corre con el guard temprano, sin crear un intervalo nuevo. Esto evita el bug clasico de 'timers acumulados corriendo en paralelo' si no se limpiaran correctamente.

3. Formateo del tiempo: de milisegundos a MM:SS

```
const minutes = Math.floor(elapsedTime / 60000);
const seconds = Math.floor((elapsedTime % 60000) / 1000);
```

```
const formatTime = (time) => String(time).padStart(2, '0');
```

- `elapsedTime` se guarda en milisegundos (no en segundos): una decision de diseno que haria mas preciso el calculo si en el futuro se quisiera mostrar decimas de segundo, aunque en esta pantalla solo se muestran minutos y segundos.
- Dividir por 60000 (60000ms = 1 minuto) da los minutos totales, truncados hacia abajo con `Math.floor`.
- El operador modulo (%) se queda con el resto (los milisegundos que no completan un minuto entero), y dividir ese resto por 1000 lo convierte a segundos.
- `padStart(2, '0')` asegura que un numero como 5 se muestre como '05' — el formato de reloj estandar.

4. Botones: Iniciar/Pausar y Reiniciar

```
onPress={() => setIsRunning((running) => !running)} // toggle
```

Un solo boton cambia de texto ('Iniciar' <-> 'Pausar') segun el valor de `isRunning`, alternando su propio valor con la misma tecnica de funcion-actualizadora vista arriba (evita depender de un valor de `isRunning` potencialmente desactualizado dentro del callback).

```
onPress={() => {  
  setIsRunning(false);  
  setElapsedTime(0);  
}}
```

El boton 'Reiniciar' detiene el cronometro y resetea el tiempo a cero en un solo tap: dos actualizaciones de estado dentro del mismo evento, que React agrupa (batching) en un solo re-render.

5. Responsividad por ancho, no solo por orientacion

Ademas de `isLandscape`, esta pantalla usa tambien 'width' del hook `useResponsiveLayout()`: `width < 360` && `styles.timerCardSmall` reduce el padding y el tamano de fuente en pantallas muy angostas. Es un caso de uso distinto al de otras pantallas de la app, que solo reaccionan a `isLandscape` (orientacion) — aca se ajusta directamente por el ancho absoluto en pixeles del dispositivo.